A Faster Way to Compute $c_t(n)$

Samuel Orellana Mateo*

March 29, 2026

Abstract

The function $c_t(n)$ is central to the enumeration of prime juggling patterns. This writeup provides a closed-form generating function for $c_t(n)$, enabling a faster algorithm to compute its values.

The function $c_t(n)$ was originally introduced by Banaian et al. [1] to establish the exact count of normal 2-ball prime juggling patterns. It was subsequently utilized as a building block to enumerate prime patterns in multiplex and colored juggling variations [2]. By definition, $c_t(n)$ involves a sum over all partitions of n into t distinct parts $p_1 > p_2 > \cdots > p_t \geq 1$:

$$c_t(n) = \sum_{\substack{p_1>\cdots>p_t\geq 1 \\ p_1+\cdots+p_t=n}} \frac{1}{t(t+1)} \prod_{i=1}^{t} \left(\frac{i+1}{i}\right)^{p_i}$$

To obtain a generating function, we change variables from the partition parts p_i to their adjacent differences. Let $\delta_t = p_t$, and $\delta_i = p_i - p_{i+1}$ for $i = 1, \ldots, t-1$. It follows that $\delta_j \geq 1$ for all $1 \leq j \leq t$. We can write $p_i = \sum_{j=i}^{t} \delta_j$. The partition sum condition then becomes:

$$\sum_{i=1}^{t} p_i = \sum_{j=1}^{t} j\delta_j = n$$

Substituting this into the product term gives a telescoping product:

$$\begin{aligned}
\prod_{i=1}^{t} \left(\frac{i+1}{i}\right)^{p_i} &= \prod_{i=1}^{t} \left(\frac{i+1}{i}\right)^{\sum_{j=i}^{t} \delta_j} \\
&= \prod_{j=1}^{t} \prod_{i=1}^{j} \left(\frac{i+1}{i}\right)^{\delta_j} \\
&= \prod_{j=1}^{t} (j+1)^{\delta_j}
\end{aligned}$$

*Duke University, Durham, NC 27708, USA. samuelorellanamateo@gmail.com

Thus, $c_t(n)$ can be rewritten as a sum over these independent differences:

$$c_t(n) = \frac{1}{t(t+1)} \sum_{\substack{\delta_1,\ldots,\delta_t \geq 1 \\ \sum_{j=1}^{t} j\delta_j = n}} \prod_{j=1}^{t} (j+1)^{\delta_j}$$

Let x be a formal variable. The constraint $\sum_{j=1}^{t} j\delta_j = n$ with the weight $(j+1)^{\delta_j}$ describe the coefficient of x^n in the product of infinite geometric series. Let $G_t(x) = \sum_{n=1}^{\infty} c_t(n)x^n$ be the generating function for a fixed t. Then:

$$G_t(x) = \frac{1}{t(t+1)} \prod_{j=1}^{t} \sum_{\delta=1}^{\infty} \Big((j+1)x^j\Big)^{\delta}$$

Evaluating the $\sum_{\delta=1}^{\infty} r^{\delta} = \frac{r}{1-r}$, we obtain the closed-form:

$$G_t(x) = \frac{1}{t(t+1)} \prod_{j=1}^{t} \frac{(j+1)x^j}{1-(j+1)x^j}$$

To compute $c_t(n)$ algorithmically, one can just compute the coefficient $[x^n]G_t(x)$.

Furthermore, we can extract a linear recurrence from this generating function to avoid the overhead of symbolic polynomial expansion. Let $H_t(x) = t(t+1)G_t(x)$, so that:

$$H_t(x) = \prod_{j=1}^{t} \frac{(j+1)x^j}{1-(j+1)x^j}$$

We can define $H_t(x)$ recursively:

$$H_t(x) = H_{t-1}(x) \frac{(t+1)x^t}{1-(t+1)x^t}$$

Multiplying both sides by the denominator gives:

$$H_t(x)\big(1-(t+1)x^t\big) = H_{t-1}(x)(t+1)x^t$$

By extracting the coefficient of x^n, which we denote as $h_t(n) = [x^n]H_t(x)$, this becomes:

$$h_t(n) = (t+1)\Big[h_{t-1}(n-t) + h_t(n-t)\Big]$$

with the base case $h_0(0) = 1$ and all other initial values set to 0. The final count is then recovered by $c_t(n) = \frac{h_t(n)}{t(t+1)}$. This reduces the computation to a $O(n\sqrt{n})$ dynamic programming algorithm that relies only on integer addition and multiplication.

References

[1] Esther Banaian, Steve Butler, Christopher Cox, Jeffrey Davis, Jacob Landgraf, and Scarlitte Ponce. Counting Prime Juggling Patterns. *Graphs and Combinatorics*, 32(5):1675–1688, Sep 2016. doi:10.1007/s00373-016-1711-1.

[2] Steve Butler, Vera Choi, Joel Jeffries, Nina McCambridge, Asia Morgenstern, and Samuel Orellana Mateo. Enumerating Prime Patterns in Juggling Variations. *arXiv preprint arXiv:2603.17284*, 2026. https://arxiv.org/abs/2603.17284.